

Creative Network



Design Institute

MCU/Raspberry Pi : GPIO와 프로그래밍

Creative Network

Design Institute

목차

1. GPIO 소개
2. 라이브러리 설치
3. 시스템 개발 방법

1.GPIO 소개

학습목표	● GPIO가 무엇인지 학습한다. ● 라즈베리 파이의 핀 맵을 확인한다.
------	--

- **필요 지식**

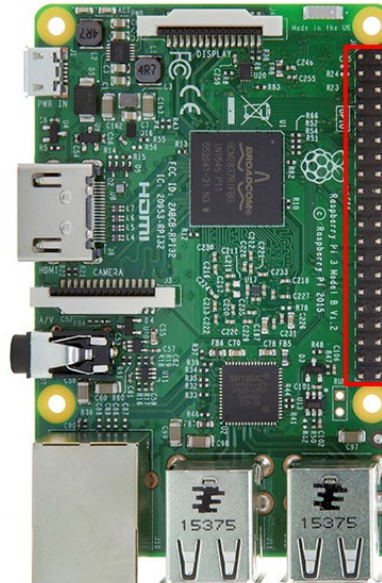
- **GPIO 개요**

GPIO 개요

- 범용으로 사용되는 입출력 포트
- 임베디드 시스템의 여러가지 기능을 위한 주변장치 및 소자들을 동작시키고 그것들이 원하는 방식으로 인터페이스를 하기 위해 적절한 신호를 전달하며, 설계자가 원하는 동작을 수행하기 위하여 포트를 입력 및 출력으로 구성
- 라즈베리 파이 보드 상에 40개의 핀들이 있다.
- 외부와 인터페이스가 가능
- GPIO 기능 이외에도 다른 장비와의 통신을 위한 SPI, I2C, UART 기능을 가짐

GPIO 개요

- GPIO Pin : 26개 (전용 핀 : 13개)
- SPI Pin : 9개 (SPI0, SPI1)
- I2C Pin : 2개 (I2C1)
- Reserved Pin : 2개 (I2C ID EEPROM)
- 5V PWR Pin : 2개
- 3.3V PWR Pin : 2개
- GND Pin : 8개



Raspberry Pi 3 GPIO Header

Pin#	NAME	NAME	Pin#
01	3.3v DC Power	DC Power 5v	02
03	GPIO02 (SDA1 , I ² C)	DC Power 5v	04
05	GPIO03 (SCL1 , I ² C)	Ground	06
07	GPIO04 (GPIO_GCLK)	(TXD0) GPIO14	08
09	Ground	(RXD0) GPIO15	10
11	GPIO17 (GPIO_GEN0)	(GPIO_GEN1) GPIO18	12
13	GPIO27 (GPIO_GEN2)	Ground	14
15	GPIO22 (GPIO_GEN3)	(GPIO_GEN4) GPIO23	16
17	3.3v DC Power	(GPIO_GEN5) GPIO24	18
19	GPIO10 (SPI_MOSI)	Ground	20
21	GPIO09 (SPI_MISO)	(GPIO_GEN6) GPIO25	22
23	GPIO11 (SPI_CLK)	(SPI_CE0_N) GPIO08	24
25	Ground	(SPI_CE1_N) GPIO07	26
27	ID_SD (I ² C ID EEPROM)	(I ² C ID EEPROM) ID_SC	28
29	GPIO05	Ground	30
31	GPIO06	GPIO12	32
33	GPIO13	Ground	34
35	GPIO19	GPIO16	36
37	GPIO26	GPIO20	38
39	Ground	GPIO21	40

2. 라이브러리 설치

학습목표	● GPIO 라이브러리인 "wiringPi"를 설치한다.
------	---

- **필요 지식**

- GPIO 라이브러리
- wiringPi 설치

GPIO 라이브러리



- GPIO를 제어하기 위해 C, C++, JAVA, Python 등이 있다.
- 각 언어에 따라 사용자들이 라이브러리를 작성 및 공유
- 그 중 본 교재에서는 C를 사용하여 GPIO를 제어하기 위해 wiringPi를 사용한다.
- wiringPi는 GPIO접근 라이브러리
- 라즈베리 파이 안에서 사용되는 BCM2835와 BCM2836을 위한 C로 작성됨
- GNU LGPLv3 라이선스 아래에서 릴리즈
- C와 C++, 그리고 많은 다른 언어를 사용됨
- 아두이노의 "wiring"시스템을 사용해온 사람들에게 매우 친근하게 디자인됨

wiringPi 설치[1/3]



- Git를 사용하기 위해 Git를 설치
- 인터넷이 연결되어 있으면 다음의 명령어 입력
- Git를 설치하는 명령어

->sudo apt-get install git-core

```
pi@raspberrypi:~ $ sudo apt-get install git-core
Reading package lists... Done
Building dependency tree
Reading state information... Done
git-core is already the newest version.
0 upgraded, 0 newly installed, 0 to remove and 4 not upgraded.
pi@raspberrypi:~ $
```

- wiringPi 소스코드 다운로드
 - git clone https://github.com/WiringPi/WiringPi
- wiringPi 빌드
 - cd WiringPi
 - ./build

wiringPi 설치[2/3]



- wiringPi 설치를 확인한다.
 - gpio -v 를 입력
 - gpio 버전과 라즈베리 파이 등의 버전을 확인할 수 있다.

```
pi@raspberrypi:~/wiringPi $ gpio -v
gpio version: 2.32
Copyright (c) 2012-2015 Gordon Henderson
This is free software with ABSOLUTELY NO WARRANTY.
For details type: gpio -warranty

Raspberry Pi Details:
  Type: Pi 3, Revision: 02, Memory: 1024MB, Maker: Sony
  * Device tree is enabled.
  * This Raspberry Pi supports user-level GPIO access.
    -> See the man-page for more details
    -> ie. export WIRINGPI_GPIOMEM=1
pi@raspberrypi:~/wiringPi $
```

wiringPi 설치[3/3]



- 라즈베리 파이 핀 맵을 확인 할수 있다.
 - gpio readall 를 입력
 - gpio핀 번호를 확인 및 상태를 확인하므로 명령어를 숙지하는 것이 좋다.

```
pi@raspberrypi:~/wiringPi $ gpio readall
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| BCM | wPi |   Name   | Mode | V | Physical | V | Mode |   Name   | wPi | BCM |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
|      |      | 3.3v     |      |   | 1 || 2 |      |      | 5v       |      |      |
| 2 | 8 | SDA.1    | IN   | 1 | 3 || 4 |      |      | 5V       |      |      |
| 3 | 9 | SCL.1    | IN   | 1 | 5 || 6 |      |      | 0v       |      |      |
| 4 | 7 | GPIO. 7   | IN   | 1 | 7 || 8 | 0 | IN   | TxD     | 15 | 14 |
|      |      | 0v       |      |   | 9 || 10 | 1 | IN   | RxD     | 16 | 15 |
| 17 | 0 | GPIO. 0   | IN   | 1 | 11 || 12 | 1 | IN   | GPIO. 1 | 1 | 18 |
| 27 | 2 | GPIO. 2   | IN   | 1 | 13 || 14 |      |      | 0v       |      |      |
| 22 | 3 | GPIO. 3   | IN   | 1 | 15 || 16 | 1 | IN   | GPIO. 4 | 4 | 23 |
|      |      | 3.3v     |      |   | 17 || 18 | 1 | IN   | GPIO. 5 | 5 | 24 |
| 10 | 12 | MOSI     | ALT0 | 1 | 19 || 20 |      |      | 0v       |      |      |
| 9 | 13 | MISO     | ALT0 | 1 | 21 || 22 | 1 | IN   | GPIO. 6 | 6 | 25 |
| 11 | 14 | SCLK     | ALT0 | 0 | 23 || 24 | 1 | OUT  | CE0     | 10 | 8 |
|      |      | 0v       |      |   | 25 || 26 | 1 | OUT  | CE1     | 11 | 7 |
| 0 | 30 | SDA.0    | IN   | 1 | 27 || 28 | 1 | IN   | SCL.0   | 31 | 1 |
| 5 | 21 | GPIO.21  | IN   | 1 | 29 || 30 |      |      | 0v       |      |      |
| 6 | 22 | GPIO.22  | IN   | 1 | 31 || 32 | 1 | IN   | GPIO.26 | 26 | 12 |
| 13 | 23 | GPIO.23  | IN   | 1 | 33 || 34 |      |      | 0v       |      |      |
| 19 | 24 | GPIO.24  | IN   | 0 | 35 || 36 | 1 | IN   | GPIO.27 | 27 | 16 |
| 26 | 25 | GPIO.25  | IN   | 0 | 37 || 38 | 0 | IN   | GPIO.28 | 28 | 20 |
|      |      | 0v       |      |   | 39 || 40 | 0 | IN   | GPIO.29 | 29 | 21 |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| BCM | wPi |   Name   | Mode | V | Physical | V | Mode |   Name   | wPi | BCM |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
pi@raspberrypi:~/wiringPi $
```

3.시스템 개발 방법

학습목표
● 라즈베리 파이 프로그램 작성에 필요한 에디터를 학습 ● 프로그램을 작성하고, 빌드하고, 실행하는 방법을 학습

● 필요지식

- 프로그램 작성
- 프로그램 빌드
- 프로그램 실행

프로그램 작성



- 텍스트 기반 프로그래밍 언어를 작성하기 위해서는 기본적으로 텍스트 에디터가 필요
- 라즈비안에 설치된 유닉스 계열 텍스트 에디터를 사용
- 보통 vi 에디터나 GNU nano 에디터를 많이 사용
- vi 에디터는 강력한 기능을 제공하지만 배우기 어려움
- nano 에디터는 기능은 단순하지만 편리한 편집기능을 제공
- 본 교재는 nano에디터를 이용

● nano 에디터

- 다른 텍스트 에디터들에 비해 가볍고 사용하기 쉬움
- 상대적으로 다양한 기능을 지원하지 않음
- 문서의 내용 구분을 쉽게 하기 위해 색상 등을 기본적으로 제공
- 실행 방법은 nano [파일명]
 - 파일을 생성할 때에도 같은 방식으로 명령을 입력
 - Ctrl+X키를 눌러 에디터를 종료 및 저장
- Ctrl+G 키를 눌러 도움말을 볼 수 있음

프로그램 작성



● 소스 코드

- 다음 명령어를 입력하여 nano에디터를 이용한 파일을 생성
- 소스 코드를 작성('₩'는 백슬레시' \ '와 같은 표현)
 - nano helloworld.c

Program Source	Description
#include <stdio.h>	표준입출력 헤더파일 포함
int main (void)	메인 함수
{	
printf("Hello World₩n");	"Hello World" 문장을 모니터에 출력
return 0;	
}	

```
#include <stdio.h>

int main (void)
{
    printf("Hello World₩n");
    return 0;
}
```

- 소스 코드 작성이 완료 되면 에디터를 종료하며 내용을 저장

● 빌드하는 법

- 빌드를 하면 프로그램을 실행 할 수 있는 실행파일이 생성
- 빌드 과정은 컴파일과 링크로 나뉨
- 문법에 오류가 있는 경우 경고나 에러를 발생하고 중지됨
- 기본 컴파일러로 gcc를 사용하며 빌드 명령은 다음과 같음

-> gcc -o helloworld helloworld.c

```
pi@raspberrypi:~ $ gcc -o helloworld helloworld.c
```

- gcc의 옵션으로 -o를 사용, 이는 실행파일에 대한 이름 부여
- 다양한 옵션을 확인하려면 gcc --help 입력

● wiringPi 라이브러리를 포함해서 컴파일 하는 방법

- gcc -o [실행파일이름] [소스코드파일] -lwiringPi

프로그램 실행

- 프로그램 빌드를 마치면 실행 파일이 생성됨
- 쉘에 다음과 같이 입력하면 프로그램을 실행 할 수 있음

➤ ./helloworld

```
pi@raspberrypi:~ $ ./helloworld  
Hello World
```

- ./[실행파일 이름]